Table of Contents & Reference Index

Golden Rules of Complexity by Constraint ($1.0\text{s} \approx 10^8$ ops | 256MB)

- $N \leq 11$: $O(N!), O(N^2 \cdot 2^N)$ (Permutations, TSP) | $N \leq 22$: $O(2^N), O(N \cdot 2^N)$ (Bitmask DP, Submasks $O(3^N)$)
- $N \leq 500$: $O(N^3)$ (Floyd-Warshall, Matrix Mult) | $N \leq 5000$: $O(N^2)$ (2D DP, All-pairs sweeps)
- $N \leq 2 \cdot 10^5$: $O(N\sqrt{N}), O(N \log^2 N)$ (Mo's, Sqrt) | $N \leq 10^6$: $O(N \log N, N)$ (Sorting, Trees, Fenwick, DSU)
- $N \leq 10^8$: $O(N)$ (Linear Sieve, Two Pointers, Kadane) | $N \geq 10^9$: $O(\sqrt{N}, \log N, 1)$ (Trial div, BS on answer, Math)

Memory Budget Rules (256MB Contest Cap) & Quick Bit Bounds

- **Array Allocation Limits**: `int` $\leq 5.0 \cdot 10^7$ | `long long` $\leq 2.5 \cdot 10^7$ | 2D `int[5000][5000]` $= 100\text{MB}$ SAFE
- **Tree/Map Overhead**: `std::set` / `std::map` $\leq 4.0 \cdot 10^6$ elements (RB-Tree node overhead ≈ 32--48 bytes/elem CAVEAT)
- **Powers of 2**: `1 << 30` $= 1.07 \times 10^9$ | `1LL << 60` $= 1.15 \times 10^{18}$ | `LLONG_MAX` $\approx 9.22 \times 10^{18}$ | `__int128` $\approx 1.7 \times 10^{38}$

Primitive Types (x64), Precision & Contest Constants

- **`int`** (32-bit): $[-2.14 \cdot 10^9, 2.14 \cdot 10^9]$ | **`uint`**: $[0, 4.29 \cdot 10^9]$ | **`ll`**: $[-9.22 \cdot 10^{18}, 9.22 \cdot 10^{18}]$ | **`ull`**: $[0, 1.84 \cdot 10^{19}]$
- **Floating Precision**: `float` (≈ 7 digits) | `double` (≈ 15--17 digits) | `long double` (≈ 18--19 digits, 80/128-bit)
- **Standard Constants**: `INF = 1e18` (SAFE: `INF + INF < LLONG_MAX`), `MOD = 1e9 + 7`, `998244353`, `PI = acos(-1.0)`
- **ASCII Bitwise Tricks**: `'a' = 97`, `'A' = 65`, `c ^ 32` (toggle case), `'0' = 48`, `c - '0'` (char to digit)

Essential Math Formulas & Pre-Submission Traps

- **Sums**: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ | $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$ | $\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$ | $\lceil a/b \rceil = \frac{a+b-1}{b}$
- **Progressions**: **AP**: $a_n = a_1 + (n-1)d$, $S_n = \frac{n(a_1+a_n)}{2} = \frac{n(2a_1+(n-1)d)}{2}$ | **GP**: $a_n = a_1 r^{n-1}$, $S_n = a_1 \frac{r^n-1}{r-1}$ $(r \neq 1)$, $S_\infty = \frac{a_1}{1-r}$ $(|r| < 1)$
- **Traps**: OVERFLOW `1LL*a*b` & `1ULL<<k` | MOD `(a - b%MOD + MOD)%MOD` | MULTI-TC Reset all globals in `while(tc--)` | SET Member `s.lower_bound(x)` $(O(\log N))$ | MULTISET `ms.erase(ms.find(x))` for single instance.

1. C++ Competitive Template, Fast I/O & PBDS

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp> // PBDS: policy-based data structs
#include <ext/pb_ds/tree_policy.hpp>     // req for order statistics (ordered_set)
using namespace std;
using namespace __gnu_pbds; // PBDS: ordered_set & gp_hash_table

/* TLE Pragmas & Caveats (uncomment w/ care):
#pragma GCC optimize("O3,unroll-loops")          // Ofast has fast-math (breaks float signs: -0.0+0.0=-0)
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt") // SIMD + HW bit ops (bmi/lzcnt/popcnt; err on old GCC)
*/

using ll = long long; using ull = unsigned long long; using ld = long double;
using pii = pair<int, int>; using pll = pair<ll, ll>;

// --- PBDS Policy-Based Data Structures ---
// 1. ordered_set: find_by_order(k) (k-th min, 0-idx) & order_of_key(x) (cnt < x) in O(log N)
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
// ordered_multiset trick (duplicates allowed): use pair<T, int> with unique id

// 2. gp_hash_table: 3x-5x faster open-addr hash map (replaces unordered_map)
// gp_hash_table<int, int> fast_map;

#define endl '\n'
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define gs(n) ((n * (n + 1)) >> 1)
#define pb push_back   // safe w/ braces {a,b}; avoids explicit ctor calls
#define eb emplace_back // in-place v.eb(a,b) w/o temp copies (faster for structs)
#define F first
#define S second
#define sz(x) (int)(x).size()
#define yn(x) (cout << ((x) ? "YES\n" : "NO\n"))

// Execution Timer (for local benchmarking):
// auto start_time = chrono::high_resolution_clock::now();
// auto duration = chrono::duration_cast<chrono::milliseconds>(chrono::high_resolution_clock::now() -
start_time).count();

void solve() {
    return; // solution logic
}

int main() {
    ios::sync_with_stdio(0); cin.tie(0);
    // freopen("input.txt", "r", stdin); freopen("output.txt", "w", stdout);
    int tc = 1;
    // cin >> tc;
    while (tc--) solve();
    return 0;
}
```

String Hashing (Polynomial Rolling Hash) ($O(N)$ Build, $O(1)$ Query)

```cpp
// Single-mod polynomial hash w/ prefix queries (use 2 instances w/ diff base/mod vs collisions): O(N)
build
struct Hash {
    vector<ll> pref, pw; ll base, mod;
    Hash(const string& s, ll base = 131, ll mod = 1e9 + 7) : base(base), mod(mod) { // O(N)
        int n = s.size(); pref.assign(n + 1, 0); pw.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            pref[i + 1] = (pref[i] * base + s[i]) % mod;
            pw[i + 1] = pw[i] * base % mod;
        }
    }
    ll get(int l, int r) { return ((pref[r + 1] - pref[l] * pw[r - l + 1]) % mod + mod) % mod; } // O(1)
};
```

2. Data Types, Limits & I/O Manipulation

Numeric Limits & Floating Precision

```cpp
#include <limits>
#include <iomanip>

int max_i = numeric_limits<int>::max();          // 2,147,483,647
ll min_ll = numeric_limits<ll>::min();           // -9,223,372,036,854,775,808LL
ull max_u = numeric_limits<ull>::max();          // 18,446,744,073,709,551,615ULL
double min_pos = numeric_limits<double>::min();  // smallest positive double (> 0)
double lowest_d = numeric_limits<double>::lowest(); // most negative double
double inf = numeric_limits<double>::infinity();    // 1.0/0.0, exp(1000), log(0) -> -inf
double nan_val = numeric_limits<double>::quiet_NaN(); // 0.0/0.0, sqrt(-1.0), inf-inf, 0.0*inf
bool is_inf = isinf(inf); bool is_nan = isnan(nan_val); // <cmath> check functions

// Fixed precision & leading zero formatting (<iomanip>):
cout << fixed << setprecision(9) << 3.1415926535; // 3.141592654 (persistent)
cout << setfill('0') << setw(2) << 5;             // "05" (setw applies ONLY to next item)
```

`getline` & Buffer Management (`cin.ignore`)

```cpp
string line;
getline(cin, line); // reads entire line w/ spaces

// Caveat: avoid mixing cin >> & getline; if mixed, cin.ignore() is mandatory:
int n; cin >> n;
cin.ignore();       // flushes leftover '\n' from buffer
getline(cin, line);
```

128-bit Integer Fast I/O ($O(\text{digits})$)

```cpp
void print128(__int128 n) {
    if (n < 0) { cout << '-'; n = -n; }
    if (n > 9) print128(n / 10);
    cout << (char)('0' + (n % 10));
}
__int128 read128() {
    __int128 x = 0; int f = 1; char ch = cin.get();
    while (ch < '0' || ch > '9') { if (ch == '-') f = -1; ch = cin.get(); }
    while (ch >= '0' && ch <= '9') { x = x * 10 + ch - '0'; ch = cin.get(); }
    return x * f;
}
```

Standard Math Functions & Float Comparisons ($O(1)$)

```cpp
min({a, b, c, d}); max({a, b, c, d}); // min/max of list: O(k)
round(1.45); // 1 | ceil(1.2); // 2 | floor(1.8); // 1 | trunc(-4.5); // -4 (toward 0)
// Integer Ceil Div for a, b > 0: (a + b - 1) / b | Floor: a / b
sqrt(x); sqrtl(x); cbrt(x); hypot(dx, dy); pow(base, exp); powl(base, exp); pow(p, 1.0 / n); // nth root
// Math Constants: M_PI (3.1415926535...), M_E (2.7182818284...), M_SQRT2 (1.41421356...)

// Fast Integer Reader (via getchar_unlocked & Ariel Parra's x10 bit-hack):
inline int fast_read_int() {
    int x = 0, f = 1; char ch = getchar_unlocked();
    while (ch < '0' || ch > '9') { if (ch == '-') f = -1; ch = getchar_unlocked(); }
    while (ch >= '0' && ch <= '9') { x = (x << 3) + (x << 1) + (ch - '0'); ch = getchar_unlocked(); }
    return x * f;
}

// Float Comparison & Radian <-> Degree Conversions:
const double EPS = 1e-9;
inline bool d_eq(double a, double b) { return abs(a - b) < EPS; }
inline double deg2rad(double d) { return d * M_PI / 180.0; }
inline double rad2deg(double r) { return r * 180.0 / M_PI; }
```

3. Bit Manipulation & `std::bitset`

Bitwise Operators & Fundamental Hacks ($O(1)$)

- `x & 1` : check odd (1) / even (0) | `a ^= b; b ^= a; a ^= b;` : in-place swap
- `1LL << k` : 2^k (`1ULL << k` for unsigned) | `x >> k` : $x/2^k$ (integer division)
- `x | (1LL << k)` : set k-th bit | `x & ~(1LL << k)` : clear k-th bit
- `x ^ (1LL << k)` : flip k-th bit | `(x >> k) & 1` : test k-th bit (0 or 1)
- `x & -x` : isolate lowest set bit (LSB) | `x & (x - 1)` : clear lowest set bit (removes rightmost 1)
- `(x > 0) && !(x & (x - 1))` : 1 if x is a power of 2
- `ch | ' '` : tolower | `ch & '_'` : toupper | `ch ^ ' '` : toggle case (ASCII bit hacks)

GCC Built-in Bit Functions ($O(1)$) (32-bit & 64-bit `ll`)

```
__builtin_popcount(x);   __builtin_popcountll(x); // count 1-bits
__builtin_clz(x);        __builtin_clzll(x);      // count leading 0s (undef if x=0)
__builtin_ctz(x);        __builtin_ctzll(x);      // count trailing 0s (undef if x=0)
__builtin_parity(x);     __builtin_parityll(x);   // 1 if odd cnt of 1s, else 0
__builtin_ffs(x);        __builtin_ffsll(x);      // 1 + idx of lowest set bit
__lg(x);                 __lg(x);                 // floor(log2(x)), highest set bit
```

Binary $\leftrightarrow$ Integer Conversions ($O(N)$)

```
// int2bin & bin2int in O(N) (by Ariel Parra):
vector<int> int2bin(ll num, int n) {
    vector<int> v(n);
    for (int i = 0; i < n; ++i) v[i] = (num >> (n - 1 - i)) & 1; // MSB first (use >> i for LSB)
    return v;
}
ll bin2int(const vector<int>& v) {
    ll res = 0;
    for (int b : v) res = (res << 1) | (b & 1);
    return res;
}
ll bin2int(const string& s) { return stoll(s, nullptr, 2); }
```

Iterate All Submasks of a Mask $O(3^N)$

```
// Iter submasks s of mask m in O(2^popcount(m)):
for (int s = m; s > 0; s = (s - 1) & m) // process submask s
// Total across all masks = O(3^N). Handle s = 0 after loop.
```

`std::bitset<N>` Usage ($O(N/64)$)

```
#include <bitset>
const int N = 64;
bitset<N> b;                        // all 0s: O(N/64)
bitset<N> b1(12345ULL);             // from int: O(1) | bitset<N> b2("11010101"); // str
bitset<N> b3(0b10110);              // bin literal

b.set(5);       b.set();            // O(1) single / O(N/64) all
b.reset(5);     b.reset();          // O(1) single / O(N/64) all
b.flip(2);      b.flip();           // O(1) single / O(N/64) all
bool v = b.test(3); int cnt = b.count(); // bounds checked test / popcount
bool any_set = b.any(); bool all_set = b.all(); bool none_set = b.none();

string s_bits = b.to_string();      // to bin str: O(N) | ull val = b.to_ullong(); // O(1)
b = b1 & b2; b |= b3; b ^= b1;      // bitwise ops in O(N/64)

// Gosper's Hack: iterate all subsets of size k from n in O(binom(n, k)):
void gospers_hack(int k, int n) {
    int mask = (1 << k) - 1;
    while (mask < (1 << n)) {
        // process mask: O(1) per subset
        int c = mask & -mask, r = mask + c;
        mask = (((r ^ mask) >> 2) / c) | r;
    }
}
```

4. String Manipulation & Parsing

Standard String Operations ($O(\text{len})$ / $O(1)$)

```
string s = "Hello World";
s.size(); s.length(); s.empty(); s.clear(); // O(1)
// Substr & Search:
string sub = s.substr(0, 5); // "Hello": O(len) | size_t pos = s.find("World"); // O(N*M)
size_t rpos = s.rfind("l");  // 9 (last): O(N*M) | if (pos != string::npos) { /* found */ }
// Find ALL occurrences (search past each prior match): O(N*M) total
for (size_t p = s.find('l'); p != string::npos; p = s.find('l', p + 1)) { /* p: match pos */ }
// Modification & Concatenation:
s.replace(6, 5, "C++");      // "Hello C++": O(N) | s.erase(5, 1); // "HelloC++": O(N)
s.insert(5, " ");            // "Hello C++": O(N) | s.push_back('!'); s.pop_back(); // O(1)
s.append("!");                // O(len) amortized | string s3 = s + " Bye"; // O(N + M), new alloc
// Numeric, Char Conversions & ASCII Loop:
int a = stoi("123"); ll b = stoll("1234567890123"); double c = stod("3.14"); // O(len)
ll val = strtoll(s.c_str(), nullptr, 16); // Base 16 or 8 or 2
string str_num = to_string(12345); int digit = ch - '0'; char char_digit = digit + '0';
for (char c = 'a'; c <= 'z'; ++c) { /* iterate 'a' to 'z' in ASCII order */ }
// GCC Case Ranges (spaces required around ...): case 'a' ... 'z': case '0' ... '9':
const char* cs = s.c_str(); // C-string: O(1) | <cctype>: isalpha(ch); isdigit(ch); isalnum(ch);
ch = tolower(ch); ch = toupper(ch);
```

StringStream Tokenization & Splitting ($O(N)$)

```
#include <sstream>
// Split line into words: O(N)
string text = "competitive programming club gallos 2026";
stringstream ss(text); string word;
while (ss >> word) { /* word: "competitive", "programming", ... */ }

// Split by delim (e.g. CSV w/ commas): O(N)
string csv = "apple,banana,cherry,grape";
stringstream ss2(csv); string item;
while (getline(ss2, item, ',')) { /* item: "apple", "banana", ... */ }
```

Palindrome Check, Rotations & KMP ($O(N)$ / $O(N^2)$)

```
// Palindrome check (ignore non-alnum): O(N)
bool isPalindrome(const string& s) {
    int l = 0, r = (int)s.size() - 1;
    while (l < r) {
        while (l < r && !isalnum(s[l])) ++l;
        while (l < r && !isalnum(s[r])) --r;
        if (tolower(s[l++]) != tolower(s[r--])) return false;
    }
    return true;
}

// Circular String Rotations (Left & Right shifts by Ariel Parra): O(N^2)
int n = s.size(); vector<string> v_left(n), v_right(n);
for (int i = 0; i < n; ++i) {
    v_left[i] = s.substr(i, n - i) + s.substr(0, i);     // left: O(N)
    v_right[i] = s.substr(n - i, i) + s.substr(0, n - i); // right: O(N)
}
// In-place rotation by k: rotate left / right in O(N) time & O(1) space
rotate(s.begin(), s.begin() + (k % n), s.end());
rotate(s.rbegin(), s.rbegin() + (k % n), s.rend());

// KMP Prefix Function pi[i] = length of longest proper border: O(N)
vector<int> prefix_function(const string& s) {
    int n = s.size(); vector<int> pi(n, 0);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}
```

5. Sorting, Comparators & Coordinate Compression

STL Sorting Functions ($O(N \log N)$)

```
vector<int> v = {5, 2, 8, 1, 9};
sort(all(v));                           // Asc: O(N log N)
sort(rall(v));                          // Desc: O(N log N)
stable_sort(all(v));                    // Preserves relative order: O(N log^2 N)
nth_element(v.begin(), v.begin() + k, v.end()); // Places k-th elem in O(N)
```

Custom Comparators & Lambdas ($O(1)$ per op)

```
struct Item {
    int id, val, weight;
};

// 1. Lambda comparator: O(1)
sort(all(items), [](const Item& a, const Item& b) {
    if (a.val != b.val) return a.val > b.val; // higher val first
    return a.weight < b.weight;               // lighter wt tie-breaker
});

// 2. Struct operator< (Overloading): O(1)
struct Point {
    int x, y;
    bool operator<(const Point& other) const {
        return tie(x, y) < tie(other.x, other.y);
    }
};

// 3. Custom Comparator Struct (4 std::set / std::priority_queue): O(1)
struct CompareItems {
    bool operator()(const Item& a, const Item& b) const {
        return a.val < b.val; // Max-heap in pq, Asc in set
    }
};
```

Coordinate Compression & Inversion Counting ($O(N \log N)$)

```
// 1. Coordinate Compression in O(N log N):
vector<int> vals = v; sort(all(vals));
vals.erase(unique(all(vals)), vals.end()); // rem dups
auto get_compressed = [&](int x) -> int {
    return lower_bound(all(vals), x) - vals.begin(); // O(log N) 0-idx rank
};
for (int &x : v) x = get_compressed(x); // in-place compress

// 2. Inversion Count (Merge Sort) in O(N log N):
ll merge_count(vector<int>& a, int l, int r) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    ll inv = merge_count(a, l, mid) + merge_count(a, mid + 1, r);
    vector<int> tmp; int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) tmp.push_back(a[i++]);
        else { tmp.push_back(a[j++]); inv += (mid - i + 1); }
    }
    while (i <= mid) tmp.push_back(a[i++]);
    while (j <= r) tmp.push_back(a[j++]);
    for (int k = 0; k < sz(tmp); ++k) a[l + k] = tmp[k];
    return inv;
}
// Caveat: Custom comparators MUST follow Strict Weak Ordering (use <, NEVER <=).
```

Pollard's Rho (Large Integer Factorization) ($O(n^{1/4})$ per Factor)

```
// Nontrivial factor of composite n (reuses mulmod/isPrime64 from Number Theory I): O(n^(1/4))
ull pollard(ull n) {
    auto f = [n](ull x) { return (mulmod(x, x, n) + 1) % n; };
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = mulmod(prd, max(x, y) - min(x, y), n))) prd = q;
        x = f(x); y = f(f(y));
    }
    return __gcd(prd, n);
}
```

6. Search: Binary & Ternary Search

STL Binary Search & Bounds Idioms ($O(\log N)$)

Array/vector must be sorted:

```
vector<int> v = {1, 3, 3, 5, 7, 8};
auto it1 = lower_bound(all(v), 3); // iterator to 1st elem >= 3: O(log N)
auto it2 = upper_bound(all(v), 3); // iterator to 1st elem > 3: O(log N)
int cnt = upper_bound(all(v), 3) - lower_bound(all(v), 3); // count of 3s (2)
bool exists = binary_search(all(v), 5); // true: O(log N)
int idx = lower_bound(all(v), 3) - v.begin(); // 0-based index (1)
```

Binary Search on Monotonic Answer ($O(\log(\text{range}) \cdot T_{\text{check}})$)

```
// 1. Minimize: Find First X in [low, high] where check(X) is TRUE:
ll low = 0, high = 1e18, min_ans = -1;
while (low <= high) {
    ll mid = low + (high - low) / 2;
    if (check(mid)) { min_ans = mid; high = mid - 1; } // search left
    else low = mid + 1;
}

// 2. Maximize: Find Last X in [low, high] where check(X) is TRUE:
low = 0; high = 1e18; ll max_ans = -1;
while (low <= high) {
    ll mid = low + (high - low) / 2;
    if (check(mid)) { max_ans = mid; low = mid + 1; }  // search right
    else high = mid - 1;
}

// 3. Continuous BS on Real Values (100 iters -> ~1e-15 precision):
double l = 0.0, r = 1e9;
for (int iter = 0; iter < 100; ++iter) {
    double mid = l + (r - l) / 2.0;
    if (checkFloat(mid)) r = mid; else l = mid;
}
```

Discrete & Continuous Ternary Search ($O(\log_{1.5}(\text{range}))$ / $O(\text{iters})$)

```
// 1. Discrete (Integer) Ternary Search: Find min of convex/unimodal f(x)
ll ternarySearchInt(ll l, ll r) {
    while (r - l > 3) {
        ll m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
        if (f(m1) > f(m2)) l = m1; else r = m2;
    }
    ll min_val = f(l), best_x = l;
    for (ll i = l + 1; i <= r; ++i) {
        if (f(i) < min_val) { min_val = f(i); best_x = i; }
    }
    return best_x;
}

// 2. Continuous (Float) Ternary Search: 100 iters -> ~1e-15 precision
double ternarySearchDouble(double l, double r) {
    for (int iter = 0; iter < 100; ++iter) {
        double m1 = l + (r - l) / 3.0, m2 = r - (r - l) / 3.0;
        if (f(m1) > f(m2)) l = m1; else r = m2;
    }
    return (l + r) / 2.0;
}
```

7. Number Theory I: Primes, Divisors & Factorization

First 25 Primes (< 100) & Classic Sieve ($O(N \log \log N)$)

```
// 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97
vector<bool> is_prime(MAXN + 1, true); vector<int> primes;
void sieve_of_eratosthenes(int n) {
    is_prime[0] = is_prime[1] = false;
    for (int p = 2; p * p <= n; ++p)
        if (is_prime[p]) for (int i = p * p; i <= n; i += p) is_prime[i] = false;
    for (int p = 2; p <= n; ++p) if (is_prime[p]) primes.push_back(p);
}
```

Deterministic Miller-Rabin for 64-bit Integers ($O(K \log^3 n), K = 12$) 64-BIT SAFE

```
inline ull mulmod(ull a, ull b, ull m) { return (ull)((__int128)a * b % m); }
inline ull modpow(ull a, ull d, ull m) {
    ull r = 1; a %= m; while (d) { if (d & 1) r = mulmod(r, a, m); a = mulmod(a, a, m); d >>= 1; }
    return r;
}
inline bool isPrime64(ull n) { // O(12 * log n) deterministic (n < 2^64)
    if (n < 2) return false;
    for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) if (n % p == 0) return n == p;
    ull d = n - 1; int r = 0; while ((d & 1) == 0) { d >>= 1; ++r; }
    for (ull a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (a >= n) continue;
        ull x = modpow(a, d, n); if (x == 1 || x == n - 1) continue;
        bool comp = true;
        for (int i = 0; i < r - 1; ++i) { x = mulmod(x, x, n); if (x == n - 1) { comp = false; break; } }
        if (comp) return false;
    }
    return true;
}
inline ull nextPrime(ull n) { if (n < 2) return 2; ++n; if (n > 2 && (n & 1) == 0) ++n; while
(!isPrime64(n)) n += 2; return n; }
inline ull prevPrime(ull n) { if (n <= 2) return 0; if (n == 3) return 2; --n; if ((n & 1) == 0) --n; while
(n >= 2 && !isPrime64(n)) n -= 2; return n; }
```

Linear Sieve (SPF), Factorization & Divisors ($O(N)$ / $O(\log N)$ / $O(\sqrt{N})$)

```
const int MAXP = 1e7; vector<int> primes, spf(MAXP + 1);
void linear_sieve(int n = MAXP) { // O(N) build SPF & primes array
    for (int i = 2; i <= n; ++i) {
        if (spf[i] == 0) { spf[i] = i; primes.push_back(i); }
        for (int p : primes) { if (p > spf[i] || (ll)i * p > n) break; spf[i * p] = p; }
    }
}
// 1. O(log n) SPF fact | 2. O(primes) sieve fact | 3. O(sqrt(n)) standalone fact:
vector<pair<int, int>> factorize_spf(int x) {
    vector<pair<int, int>> f;
    while (x > 1) { int p = spf[x], c = 0; while (x % p == 0) { x /= p; ++c; } f.push_back({p, c}); }
    return f;
}
vector<pair<ll, int>> factorize_sieve(ll n, const vector<int>& pms) {
    vector<pair<ll, int>> f;
    for (int p : pms) {
        if (1LL * p * p > n) break;
        if (n % p == 0) { int c = 0; while (n % p == 0) { n /= p; ++c; } f.push_back({p, c}); }
    }
    if (n > 1) f.push_back({n, 1}); return f;
}
vector<pair<ll, int>> factorize(ll n) {
    vector<pair<ll, int>> f;
    for (ll d = 2; d * d <= n; ++d)
        if (n % d == 0) { int c = 0; while (n % d == 0) { n /= d; ++c; } f.push_back({d, c}); }
    if (n > 1) f.push_back({n, 1}); return f;
}
// Generate all divisors in O(d(N)) | Sum of Divisors Sieve in O(N log N):
void get_divisors(int idx, ll cur, const vector<pair<int, int>>& f, vector<ll>& divs) {
    if (idx == sz(f)) { divs.push_back(cur); return; }
    for (int i = 0; i <= f[idx].second; ++i) { get_divisors(idx + 1, cur, f, divs); cur *= f[idx].first; }
}
vector<ll> sum_divisors_sieve(int n) {
    vector<ll> s(n + 1, 0); for (int i = 1; i <= n; ++i) for (int j = i; j <= n; j += i) s[j] += i;
    return s;
}
```

8. Range Queries & Prefix Sums

1D & 2D Prefix Sums ($O(1)$ Query)

```
// 1D Pref Sum: build O(N), query in O(1)
vector<ll> buildPrefix1D(const vector<ll>& a) {
    int n = a.size();
    vector<ll> pref(n + 1, 0);
    for (int i = 0; i < n; ++i) pref[i + 1] = pref[i] + a[i];
    return pref;
}
ll query1D(const vector<ll>& pref, int L, int R) { // O(1)
    return pref[R + 1] - pref[L];
}

// 2D Pref Sum: build O(R * C), query in O(1)
vector<vector<ll>> buildPrefix2D(const vector<vector<ll>>& a) {
    int R = a.size(), C = a[0].size();
    vector<vector<ll>> p(R + 1, vector<ll>(C + 1, 0));
    for (int r = 0; r < R; ++r) {
        for (int c = 0; c < C; ++c) {
            p[r + 1][c + 1] = a[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c];
        }
    }
    return p;
}
ll query2D(const vector<vector<ll>>& p, int r1, int c1, int r2, int c2) { // O(1)
    return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
}
```

1D Difference Array (Range Updates in $O(1)$)

```
// Range update O(1), reconstruct in O(N):
struct DiffArray {
    int n;
    vector<ll> diff;
    DiffArray(int n) : n(n), diff(n + 2, 0) {}
    void add(int L, int R, ll val) { // O(1)
        diff[L] += val;
        diff[R + 1] -= val;
    }
    vector<ll> reconstruct() { // O(N)
        vector<ll> res(n);
        ll cur = 0;
        for (int i = 0; i < n; ++i) { cur += diff[i]; res[i] = cur; }
        return res;
    }
};
```

Sqrt Decomposition (Point Update $O(1)$ & Range Query $O(\sqrt{N})$)

```
struct SqrtDecomp {
    int n, block_sz;
    vector<ll> a, blocks;
    SqrtDecomp(const vector<ll>& arr) : n(arr.size()), a(arr) { // O(N) build
        block_sz = sqrt(n) + 1;
        blocks.assign(n / block_sz + 1, 0);
        for (int i = 0; i < n; ++i) blocks[i / block_sz] += a[i];
    }
    void update(int idx, ll val) { // O(1)
        blocks[idx / block_sz] += (val - a[idx]);
        a[idx] = val;
    }
    ll query(int L, int R) { // O(sqrt(N))
        ll sum = 0;
        while (L <= R && L % block_sz != 0) sum += a[L++];
        while (L + block_sz - 1 <= R) { sum += blocks[L / block_sz]; L += block_sz; }
        while (L <= R) sum += a[L++];
        return sum;
    }
};
```

9. Two Pointers & Sliding Window

Two Pointers on Sorted Array ($O(N)$)

```cpp
// Pair sum arr[i] + arr[j] == target in sorted arr: O(N)
bool twoSumSorted(const vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        int sum = a[l] + a[r];
        if (sum == target) return true;
        if (sum < target) ++l;
        else --r;
    }
    return false;
}
```

Dynamic Sliding Window ($O(N)$)

```cpp
// Max len subarr w/ sum <= X (pos elems): O(N)
int longestSubarraySum(const vector<int>& a, ll X) {
    int n = a.size(), l = 0, max_len = 0;
    ll cur_sum = 0;
    for (int r = 0; r < n; ++r) {
        cur_sum += a[r];
        while (cur_sum > X && l <= r) {
            cur_sum -= a[l++];
        }
        max_len = max(max_len, r - l + 1);
    }
    return max_len;
}
```

Variable Window: At Most K Distinct Elements ($O(N)$)

```cpp
// Cnt subarrs w/ <= K distinct elems: O(N)
ll atMostKDistinct(const vector<int>& a, int k) {
    int n = a.size(), l = 0;
    unordered_map<int, int> freq;
    ll ans = 0;
    for (int r = 0; r < n; ++r) {
        freq[a[r]]++;
        while ((int)freq.size() > k) {
            if (--freq[a[l]] == 0) freq.erase(a[l]);
            ++l;
        }
        ans += (r - l + 1); // all subarrs ending at r
    }
    return ans;
}
// Exact K distinct = atMostKDistinct(a, k) - atMostKDistinct(a, k - 1)
```

Sliding Window Minimum / Maximum (Monotonic Deque $O(N)$)

```cpp
// Min for each window of sz k in O(N):
vector<int> slidingWindowMin(const vector<int>& a, int k) {
    int n = a.size();
    deque<int> dq; // stores idx, vals in inc order
    vector<int> res;
    for (int i = 0; i < n; ++i) {
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

10. Non-Linear Structures & STL Algorithms

`std::set`, `std::multiset` & `std::map` ($O(\log N)$)

- `set` : ordered, unique, $O(\log N)$ insert/erase/find
- `multiset` : ordered, allows dups, $O(\log N)$
- `map` : ordered key-val store, $O(\log N)$

```
set<int> s; s.insert(5); s.erase(5); // O(log N)
if (s.count(5)) { /* exists */ }
auto it = s.lower_bound(5); // member s.lower_bound(x) is O(log N)

multiset<int> ms;
ms.insert(5); ms.insert(5);
ms.erase(ms.find(5)); // erases SINGLE instance of 5: O(log N)
// ms.erase(5);       // WARNING: erases ALL instances of 5

map<string, int> mp;
mp["alice"] = 100; // O(log N): overwrites if key exists
mp.insert(make_pair("bob", 90));  // O(log N): no-op if key exists
mp.insert({"carol", 95});         // same, brace-init pair
auto it_m = mp.find("alice"); // O(log N)
if (it_m != mp.end()) cout << it_m->first << ": " << it_m->second;
for (auto& [k, val] : mp) cout << k << ": " << val << " "; // iterates in key-sorted order: O(N)
```

`std::priority_queue` (Heaps: $O(\log N)$ push/pop, $O(1)$ top)

```
priority_queue<int> max_heap; // top() is max: O(1) top, O(log N) push/pop
priority_queue<int, vector<int>, greater<int>> min_heap; // top() is min
auto cmp = [](const pair<int, int>& a, const pair<int, int>& b) { return a.first > b.first;}; // min-heap
on 1st elem
priority_queue<pair<int, int>, vector<pair<int, int>>, decltype(cmp)> custom_pq(cmp);
```

Hash Tables: `map` vs `unordered_map`/`unordered_set` vs `gp_hash_table` ($O(1)$ avg)

- SAFE `map` / `set` : $O(\log N)$ guaranteed (RB-Tree), ordered keys.
- CAVEAT `unordered_map` / `unordered_set` : $O(1)$ avg, but $O(N)$ worst-case (vulnerable to anti-hash hacks on Codeforces).
- FAST `gp_hash_table` : Open-addressing, $3\times$ --$5\times$ faster than `unordered_map` (use `null_type` as the value to get a set: `gp_hash_table<int, null_type>`).

```
// Custom Safe Hash (prevents O(N) worst-case hacks on CF): O(1)
struct custom_hash {
    static ull splitmix64(ull x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(ull x) const {
        static const ull FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
// Usage: gp_hash_table<ll, int, custom_hash> safe_map; // O(1) avg

unordered_set<int> us; us.insert(5); us.erase(5); if (us.count(5)) {} // O(1) avg, same anti-hash caveat as
unordered_map
```

Useful `<numeric>` & `<algorithm>` Functions ($O(N)$)

```
// Sum/prod, iota, merge/set ops, permutations, min/max, reverse/rotate: O(N)
ll sum = accumulate(all(v), 0LL); ll prod = accumulate(all(v), 1LL, multiplies<ll>());
fill(all(v), 0); iota(all(v), 1); // fills w/ 1, 2, 3, ..., n
vector<int> mrg, inter, un, diff;
merge(all(v1), all(v2), back_inserter(mrg));
set_intersection(all(a), all(b), back_inserter(inter)); // A ∩ B: O(N + M)
set_union(all(a), all(b), back_inserter(un));           // A ∪ B: O(N + M)
set_difference(all(a), all(b), back_inserter(diff));    // A \ B: O(N + M)
sort(all(v)); do { /* perm */ } while (next_permutation(all(v)));
int min_val = *min_element(all(v)); int max_val = *max_element(all(v)); // min/max val
auto [mn_it, mx_it] = minmax_element(all(v)); // single pass O(N)
reverse(all(v)); rotate(v.begin(), v.begin() + k, v.end()); // O(N)
```

11. Combinatorics & Counting

Factorials & Binomial Coefficients ($O(N)$ Prep, $O(1)$ Query)

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

```
const int MAXC = 1e6;
const ll MOD = 1e9 + 7;
ll fac[MAXC + 1], invFac[MAXC + 1];

void init_comb(int n = MAXC) { // O(N) precalc
    fac[0] = 1;
    for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i % MOD;
    invFac[n] = mod_exp(fac[n], MOD - 2, MOD);
    for (int i = n - 1; i >= 0; --i) invFac[i] = invFac[i + 1] * (i + 1) % MOD;
}

ll nCr(int n, int r) { // O(1) query
    if (r < 0 || r > n) return 0;
    return fac[n] * invFac[r] % MOD * invFac[n - r] % MOD;
}

ll nPr(int n, int r) { // O(1) query
    if (r < 0 || r > n) return 0;
    return fac[n] * invFac[n - r] % MOD;
}

// Stars & Bars: n identical items into k distinct bins: O(1)
ll stars_and_bars(int n, int k) { return nCr(n + k - 1, k - 1); }
```

Catalan Numbers ($O(1)$)

Formula: $C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}$

Applications: Valid parens seqs of len $2n$, Binary trees w/ n nodes, Dyck paths.

```
ll catalan(int n) { // O(1) using precalculated fac/invFac
    return nCr(2 * n, n) * mod_inv_prime(n + 1, MOD) % MOD;
}
```

Derangements ($O(N)$)

Recurrence: $D_n = (n-1)(D_{n-1} + D_{n-2}), \quad D_0 = 1, D_1 = 0$

```
vector<ll> derangements(int n) { // O(N) precalculation
    vector<ll> d(n + 1); d[0] = 1; if (n >= 1) d[1] = 0;
    for (int i = 2; i <= n; ++i) d[i] = (1LL * (i - 1) * (d[i - 1] + d[i - 2])) % MOD;
    return d;
}
```

Pigeonhole Principle & Inclusion-Exclusion ($O(2^M \cdot M)$)

- **Pigeonhole Principle**: If N items put in K boxes, ≥ 1 box has $\lceil N/K \rceil$ items ($O(1)$).
- **Inclusion-Exclusion**: $|A_1 \cup \cdots \cup A_n| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \ldots$

```
// Cnt nums in [1, N] div by >= 1 prime in p[]: O(2^M * M)
ll count_divisible(ll N, const vector<ll>& primes) {
    int m = primes.size(); ll ans = 0;
    for (int mask = 1; mask < (1 << m); ++mask) {
        ll prod = 1; int cnt = 0;
        for (int i = 0; i < m; ++i) if ((mask >> i) & 1) {
            ++cnt; if (prod > N / primes[i]) { prod = N + 1; break; }
            prod *= primes[i];
        }
        ans += (cnt & 1 ? 1 : -1) * (N / prod);
    }
    return ans;
}

// Lucas' Theorem: nCr % p for large n, r when p is prime: O(p + log_p n)
ll lucas_nCr(ll n, ll r, ll p) {
    if (r == 0) return 1;
    ll ni = n % p, ri = r % p;
    if (ri > ni) return 0;
    return lucas_nCr(n / p, r / p, p) * nCr(ni, ri) % p;
}
```

12. Number Theory II: Modular Arithmetic & GCD

Euclidean Algorithm & Extended GCD ($O(\log(\min(a,b)))$)

- SAFE `std::gcd` / `std::lcm` (C++17, `<numeric>`): always returns non-negative, regardless of operand signs.
- CAVEAT `__gcd(a, b)`: non-standard GCC builtin (`<bits/stl_algobase.h>`), handles 0 fine but does **not** normalize sign — can return negative (e.g. `__gcd(-6, 4) == -2`).

```
// Prefer std::gcd/std::lcm (C++17); __gcd is a GCC extension that can return negative results
inline __int128 gcd128(__int128 a, __int128 b) { return b == 0 ? a : gcd128(b, a % b); }

// ExtGCD (not in STL): finds x, y s.t. a*x + b*y = gcd(a, b) in O(log(min(a, b)))
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1, g = extgcd(b, a % b, x1, y1);
    x = y1; y = x1 - y1 * (a / b);
    return g;
}
```

Modular Arithmetic & Inverses ($O(\log(\text{exp}))$ / $O(\log(\text{MOD}))$)

$(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m \quad | \quad (a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$

$(a-b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m \quad | \quad (a/b) \bmod m = (a \times b^{-1}) \bmod m \quad (\gcd(b,m) = 1)$

```
ll mod_exp(ll base, ll exp, ll mod) { // O(log exp)
    ll res = 1; base %= mod;
    while (exp > 0) {
        if (exp & 1) res = (__int128)res * base % mod;
        base = (__int128)base * base % mod; exp >>= 1;
    }
    return res;
}
// Mod inv (MOD prime: Fermat | coprime: ExtGCD): O(log MOD)
ll mod_inv_prime(ll b, ll mod) { return mod_exp(b, mod - 2, mod); }
ll mod_inv_general(ll a, ll mod) {
    ll x, y, g = extgcd(a, mod, x, y);
    return g == 1 ? (x % mod + mod) % mod : -1;
}
```

Euler's Totient $\phi(n)$, Segmented Sieve & CRT

```
ll phi(ll n) { // Cnt nums in [1, n] coprime to n: O(sqrt(n))
    ll res = n;
    for (ll p = 2; p * p <= n; ++p) {
        if (n % p == 0) { while (n % p == 0) n /= p; res -= res / p; }
    }
    if (n > 1) res -= res / n;
    return res;
}

// Primes in [L, R]: O((R - L + 1) log log R + sqrt(R))
vector<ll> segmentedSieve(ll L, ll R) {
    ll lim = sqrt(R); vector<bool> mark(lim + 1, true); vector<ll> primes;
    for (ll i = 2; i <= lim; ++i) if (mark[i]) { primes.push_back(i); for (ll j = i * i; j <= lim; j += i)
mark[j] = false; }
    vector<bool> is_p(R - L + 1, true);
    for (ll p : primes) for (ll j = max(p * p, (L + p - 1) / p * p); j <= R; j += p) is_p[j - L] = false;
    if (L == 1) is_p[0] = false;
    vector<ll> res;
    for (ll i = L; i <= R; ++i) if (is_p[i - L]) res.push_back(i);
    return res;
}

// Chinese Remainder Theorem (CRT) for coprime moduli: O(K log(prod))
ll crt(const vector<ll>& num, const vector<ll>& rem) {
    ll prod = 1, ans = 0;
    for (ll n : num) prod *= n;
    for (int i = 0; i < sz(num); ++i) {
        ll pp = prod / num[i];
        ans = (ans + rem[i] * mod_inv_general(pp, num[i]) % prod * pp) % prod;
    }
    return (ans + prod) % prod;
}
```

13. Linear Structures: Vector, Deque, Stack, Queue

`std::vector` & 2D Matrices ($O(1)$ amortized)

```
vector<int> v(n, 0);                // sz n init to 0: O(N)
vector<vector<int>> grid(R, vector<int>(C, 0)); // R x C matrix: O(R * C)
vector<int> v2 = v;                 // deep copy: O(N) | v1.insert(v1.end(), all(v2)); // append v2: O(M)
v.reserve(100000);                  // pre-alloc mem: O(N)
v.push_back(x);                     // O(1) amortized: safe (implicit conversions only)
// emplace_back: in-place ctor (faster for pairs: v.emplace_back(a,b); caveats w/ explicit ctors)
v.pop_back();                       // O(1)
v.insert(v.begin() + idx, x);       // insert @ pos (shifts tail): O(N)
v.resize(new_sz, 0);                // grow/shrink, pad new elems w/ 0: O(N)
v.shrink_to_fit();                  // O(N)
v.clear();                          // O(N), cap UNCHANGED; to free mem: vector<int>().swap(v);
```

`std::deque` ($O(1)$ push/pop at both ends)

Fast $O(1)$ push/pop at both ends & $O(1)$ random indexing.

```
deque<int> dq;
dq.push_back(10); dq.push_front(20); // O(1)
dq.pop_back();    dq.pop_front();     // O(1)
int front_val = dq.front(); int back_val = dq.back(); // O(1)
int elem = dq[2];                   // random access in O(1)
```

`std::stack`, `std::queue` & Monotonic Stack ($O(N)$)

```
// Stack (LIFO): st.push(1); st.pop(); st.top(); | Queue (FIFO): q.push(1); q.pop(); q.front();
// 1. Next Greater Element (NGE) in O(N):
vector<int> nextGreaterElement(const vector<int>& a) {
    int n = a.size(); vector<int> nge(n, -1); stack<int> st;
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.top()] < a[i]) { nge[st.top()] = a[i]; st.pop(); }
        st.push(i);
    }
    return nge;
}

// 2. Largest Rectangle in Histogram in O(N):
ll largestRectangleArea(const vector<ll>& h) {
    stack<int> st; ll max_a = 0; int n = h.size();
    for (int i = 0; i <= n; ++i) {
        ll cur_h = (i == n ? 0 : h[i]);
        while (!st.empty() && h[st.top()] >= cur_h) {
            ll height = h[st.top()]; st.pop();
            ll width = st.empty() ? i : i - st.top() - 1;
            max_a = max(max_a, height * width);
        }
        st.push(i);
    }
    return max_a;
}
```

Circular Array Traversal & Index Arithmetic ($O(1)$)

```
// Advance k steps CW: (cur + k) % n | CCW: (cur - k % n + n) % n in O(1)
// Shortest cyclic dist: min(abs(i - j), n - abs(i - j)) in O(1)
for (int i = idx1; i != idx2; i = (i + 1) % n) {}     // CW (Right): O(steps)
for (int i = idx1; i != idx2; i = (i - 1 + n) % n) {} // CCW (Left): O(steps)
// Cyclic neighbors: left = (i - 1 + n) % n, right = (i + 1) % n
```

Z-Function (Pattern Matching) ($O(N)$)

```
// z[i] = length of longest common prefix of s and s[i:]: O(N)
vector<int> z_function(const string& s) {
    int n = s.size();
    vector<int> z(n); int l = 0, r = 0;
    for (int i = 1; i < n; ++i) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}
// Pattern search: run on (pattern + '#' + text); match at i where z[i] == pattern.size()
```

14. Graph Traversals: DFS, BFS & Flood Fill

Graph Representations & DFS / Bipartite Check ($O(V+E)$)

```
const int MAXV = 1e5 + 5;
vector<int> adj[MAXV]; vector<pair<int, ll>> adjW[MAXV]; // Unweighted / Weighted
bool vis[MAXV]; int color[MAXV]; // -1: uncolored, 0/1: colors

void dfs(int u) { // O(V + E)
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) dfs(v);
}
bool isBipartite(int u, int c = 0) { // O(V + E) 2-coloring
    color[u] = c;
    for (int v : adj[u]) {
        if (color[v] == -1 && !isBipartite(v, 1 - c)) return false;
        else if (color[v] == color[u]) return false;
    }
    return true;
}
```

Breadth-First Search (BFS) & Shortest Path ($O(V+E)$)

```
// Unweighted single-source shortest path: O(V + E)
vector<int> bfs_dist(int src, int n) {
    vector<int> dist(n + 1, -1); queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
    }
    return dist;
}
```

2D Grid Flood Fill & Kahn's Topological Sort ($O(R \cdot C)$ / $O(V+E)$)

```
const int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
void floodFill(vector<vector<int>>& g, int sr, int sc, int newColor) {
    int R = g.size(), C = g[0].size(), oldColor = g[sr][sc];
    if (oldColor == newColor) return;
    queue<pair<int, int>> q; q.push({sr, sc}); g[sr][sc] = newColor;
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        for (int i = 0; i < 4; ++i) {
            int nr = r + dx[i], nc = c + dy[i];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C && g[nr][nc] == oldColor) {
                g[nr][nc] = newColor; q.push({nr, nc});
            }
        }
    }
}

// Topological Sort (Kahn's Algorithm) & Cycle Detection in O(V + E):
vector<int> topoSort(int n) {
    vector<int> in(n + 1, 0), order;
    for (int u = 1; u <= n; ++u) for (int v : adj[u]) ++in[v];
    queue<int> q;
    for (int i = 1; i <= n; ++i) if (in[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : adj[u]) if (--in[v] == 0) q.push(v);
    }
    return (int)order.size() == n ? order : vector<int>(); // empty if cycle
}
```

Fenwick Tree (Binary Indexed Tree): Point Update & Prefix Sum $O(\log N)$

```
struct Fenwick {
    int n; vector<ll> tree;
    Fenwick(int n) : n(n), tree(n + 1, 0) {} // O(N)
    void add(int i, ll delta) { for (; i <= n; i += i & -i) tree[i] += delta; } // O(log N)
    ll query(int i) { ll sum = 0; for (; i > 0; i -= i & -i) sum += tree[i]; return sum; } // O(log N)
    ll query(int l, int r) { return query(r) - query(l - 1); } // O(log N)
};
```

15. Shortest Paths, DSU & Minimum Spanning Tree

Dijkstra's Shortest Path ($O((V+E)\log V)$)

```
const ll INF = 1e18;
// O((V + E) log V) with priority_queue:
vector<ll> dijkstra(int src, int n) {
    vector<ll> dist(n + 1, INF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // skip outdated
        for (auto& [v, w] : adjW[u]) {
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
        }
    }
    return dist;
}
```

Disjoint Set Union (DSU) ($O(\alpha(N)) \approx O(1)$ per op)

```
// DSU with Path Compression & Union by Size:
struct DSU {
    vector<int> parent, sz; int num_sets;
    DSU(int n) : parent(n + 1), sz(n + 1, 1), num_sets(n) { iota(all(parent), 0); }
    int find(int i) { return parent[i] == i ? i : parent[i] = find(parent[i]); }
    bool unite(int i, int j) {
        int root_i = find(i), root_j = find(j);
        if (root_i == root_j) return false;
        if (sz[root_i] < sz[root_j]) swap(root_i, root_j); // union by sz
        parent[root_j] = root_i; sz[root_i] += sz[root_j]; num_sets--;
        return true;
    }
    bool same(int i, int j) { return find(i) == find(j); }
    int size(int i) { return sz[find(i)]; }
};
```

Kruskal's MST, Floyd-Warshall & Bellman-Ford ($O(E\log E)$ | $O(V^3)$ | $O(V\cdot E)$)

```
struct Edge { int u, v; ll w; bool operator<(const Edge& o) const { return w < o.w; } };
// 1. Kruskal's MST in O(E log E):
pair<ll, vector<Edge>> kruskal(int n, vector<Edge>& edges) {
    sort(all(edges)); DSU dsu(n); ll mst_cost = 0; vector<Edge> mst_edges;
    for (const auto& e : edges) {
        if (dsu.unite(e.u, e.v)) { mst_cost += e.w; mst_edges.push_back(e); }
    }
    return {mst_cost, mst_edges};
}
// 2. Floyd-Warshall All-Pairs Shortest Path in O(V^3):
void floydWarshall(int n, vector<vector<ll>>& d) {
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (d[i][k] < INF && d[k][j] < INF) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
// 3. Bellman-Ford & Negative Cycle Detection in O(V * E):
bool bellmanFord(int src, int n, const vector<Edge>& edges, vector<ll>& dist) {
    dist.assign(n + 1, INF); dist[src] = 0;
    for (int i = 1; i < n; ++i)
        for (const auto& e : edges)
            if (dist[e.u] < INF && dist[e.u] + e.w < dist[e.v]) dist[e.v] = dist[e.u] + e.w;
    for (const auto& e : edges)
        if (dist[e.u] < INF && dist[e.u] + e.w < dist[e.v]) return true; // neg cycle reachable
    return false;
}
```

N-ary & Ternary Trees (Left-Child / Right-Sibling) ($O(N)$)

```
// N-ary / Generic Tree Node (first child + next sibling):
struct NaryNode {
    int data; NaryNode *firstChild, *nextSibling;
    NaryNode(int val) : data(val), firstChild(nullptr), nextSibling(nullptr) {}
};
// Ternary Tree Node (left, center, right):
struct TernaryNode {
    int data; TernaryNode *left, *center, *right;
    TernaryNode(int val) : data(val), left(nullptr), center(nullptr), right(nullptr) {}
};
```

16. Hopcroft-Karp: Maximum Bipartite Matching

Hopcroft-Karp Algorithm ($O(E\sqrt{V})$)

```
struct HopcroftKarp { // 1-idxd; v = left node, u = right node: O(E * sqrt(V))
    int n, m; // # left, right nodes
    vector<vector<int>> adj; vector<int> left, depth, iter; vector<bool> matched;
    HopcroftKarp(int _n, int _m) : n(_n), m(_m), adj(_n + 1) {}
    void addEdge(int v, int u) { adj[v].push_back(u); } // O(1)

    bool bfs() { // O(V + E): builds layered graph, true if an augmenting path may exist
        depth.assign(n + 1, -1); queue<int> q;
        for (int v = 1; v <= n; ++v) if (!matched[v]) { depth[v] = 0; q.push(v); }
        bool hasPath = false;
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int u : adj[v]) {
                if (left[u] == -1) hasPath = true;
                else if (depth[left[u]] == -1) { depth[left[u]] = depth[v] + 1; q.push(left[u]); }
            }
        }
        return hasPath;
    }
    bool dfs(int v) { // O(E) amortized per phase: finds augmenting path via layered graph
        for (int& i = iter[v]; i < sz(adj[v]); ++i) {
            int u = adj[v][i];
            if (left[u] == -1 || (depth[left[u]] == depth[v] + 1 && dfs(left[u]))) {
                left[u] = v; matched[v] = true; return true;
            }
        }
        depth[v] = -1; return false;
    }
    int maxMatching() { // O(E * sqrt(V)): repeatedly augment by phases until none remain
        matched.assign(n + 1, false); left.assign(m + 1, -1);
        int match = 0;
        while (bfs()) {
            iter.assign(n + 1, 0);
            for (int v = 1; v <= n; ++v) if (!matched[v] && dfs(v)) ++match;
        }
        return match;
    }
};
```

17. Tree Algorithms & Lowest Common Ancestor

Tree Diameter (Longest Path in a Tree) $O(N)$

Method: 2-pass BFS/DFS to find furthest endpoints in $O(N)$.

```
pair<int, int> bfs_furthest(int src, int n) { // O(N)
    vector<int> dist(n + 1, -1);
    queue<int> q;
    dist[src] = 0; q.push(src);
    int furthest_node = src;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (dist[u] > dist[furthest_node]) furthest_node = u;
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return {furthest_node, dist[furthest_node]};
}

int getTreeDiameter(int n) { // O(N) total
    auto [nodeA, distA] = bfs_furthest(1, n);
    auto [nodeB, diameter] = bfs_furthest(nodeA, n);
    return diameter;
}
```

Tree Subtree Sizes & Depths (Tree DFS) ($O(N)$)

```
int subtree_sz[MAXV], depth[MAXV], parent_node[MAXV];

void tree_dfs(int u, int p = 0, int d = 0) { // O(N)
    subtree_sz[u] = 1;
    depth[u] = d;
    parent_node[u] = p;
    for (int v : adj[u]) {
        if (v != p) {
            tree_dfs(v, u, d + 1);
            subtree_sz[u] += subtree_sz[v];
        }
    }
}
```

Lowest Common Ancestor (Binary Lifting) & Euler Tour ($O(N \log N)$ / $O(N)$)

```
const int LOGN = 20;
int up[MAXV][LOGN];

void lca_dfs(int u, int p = 0, int d = 0) { // O(N log N) build
    depth[u] = d; up[u][0] = p;
    for (int i = 1; i < LOGN; ++i) up[u][i] = up[up[u][i - 1]][i - 1];
    for (int v : adj[u]) if (v != p) lca_dfs(v, u, d + 1);
}

int get_lca(int u, int v) { // O(log N) query
    if (depth[u] < depth[v]) swap(u, v);
    for (int i = LOGN - 1; i >= 0; --i)
        if (depth[u] - (1 << i) >= depth[v]) u = up[u][i];
    if (u == v) return u;
    for (int i = LOGN - 1; i >= 0; --i)
        if (up[u][i] != up[v][i]) { u = up[u][i]; v = up[v][i]; }
    return up[u][0];
}

// Euler Tour / Tree Flattening for Subtree Queries in O(N):
int timer = 0, tin[MAXV], tout[MAXV];
void euler_tour(int u, int p = 0) {
    tin[u] = ++timer;
    for (int v : adj[u]) if (v != p) euler_tour(v, u);
    tout[u] = timer; // Subtree of u corresponds to contiguous range [tin[u], tout[u]]
}
```

18. Dynamic Programming I: Classic Paradigms

Fibonacci (Binet $O(1)$, Memo $O(N)$, Tabulation $O(1)$ Space)

```
// 1. Binet Formula (O(1) exact for n <= 70):
ull fib_binet(int n) { return round(pow((1.0 + sqrt(5.0)) / 2.0, n) / sqrt(5.0)); }
// 2. Top-Down Memoization: O(N) time & space
vector<ll> memo(MAXN, -1);
ll fib_memo(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib_memo(n - 1) + fib_memo(n - 2);
}
// 3. Bottom-Up Tabulation (O(N) time, O(1) Space):
ull fib(int n, ull MOD = 1e9 + 7) {
    if (n <= 1) return n;
    ull a = 0, b = 1;
    for (int i = 2; i <= n; ++i) { ull c = (a + b) % MOD; a = b; b = c; }
    return b;
}
```

0/1 & Unbounded Knapsack ($O(N \cdot W)$ Space-Optimized 1D)

```
// 1. 0/1 Knapsack (iter BACKWARD w = W -> wt[i]): O(N * W) time, O(W) space
int knapsack01(int W, const vector<int>& wt, const vector<int>& val) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < sz(wt); ++i)
        for (int w = W; w >= wt[i]; --w) dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
// 2. Unbounded Knapsack (iter FORWARD w = wt[i] -> W): O(N * W) time, O(W) space
int knapsackUnbounded(int W, const vector<int>& wt, const vector<int>& val) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < sz(wt); ++i)
        for (int w = wt[i]; w <= W; ++w) dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
```

Coin Change: Minimum Coins & Total Ways ($O(N \cdot X)$)

```
const int INF = 1e9;
// 1. Min coins for amount X: O(N * X) time, O(X) space
int minCoins(int X, const vector<int>& coins) {
    vector<int> dp(X + 1, INF); dp[0] = 0;
    for (int i = 1; i <= X; ++i)
        for (int c : coins) if (i >= c && dp[i - c] != INF) dp[i] = min(dp[i], dp[i - c] + 1);
    return dp[X] == INF ? -1 : dp[X];
}
// 2. Cnt combos for amount X (unbounded knapsack): O(N * X) time, O(X) space
ll countCoinWays(int X, const vector<int>& coins, ll MOD = 1e9 + 7) {
    vector<ll> dp(X + 1, 0); dp[0] = 1;
    for (int c : coins) // coin outside loop = unordered combos
        for (int i = c; i <= X; ++i) dp[i] = (dp[i] + dp[i - c]) % MOD;
    return dp[X];
}
```

Grid Paths: Count Paths with Obstacles $O(R \cdot C)$

```
// O(R * C) time & space:
ll gridPaths(const vector<vector<int>>& g, ll MOD = 1e9 + 7) {
    int R = g.size(), C = g[0].size();
    if (g[0][0] == 1 || g[R - 1][C - 1] == 1) return 0;
    vector<vector<ll>> dp(R, vector<ll>(C, 0)); dp[0][0] = 1;
    for (int r = 0; r < R; ++r) for (int c = 0; c < C; ++c) {
        if (g[r][c] == 1) { dp[r][c] = 0; continue; }
        if (r > 0) dp[r][c] = (dp[r][c] + dp[r - 1][c]) % MOD;
        if (c > 0) dp[r][c] = (dp[r][c] + dp[r][c - 1]) % MOD;
    }
    return dp[R - 1][C - 1];
}
```

19. Dynamic Programming II: Sequences & Substrings

Longest Increasing Subsequence (LIS) in $O(N \log N)$

```
// LIS len & reconstr seq in O(N log N) time, O(N) space:
pair<int, vector<int>> getLIS(const vector<int>& a) {
    int n = a.size();
    vector<int> tails, tail_indices, parent(n, -1);
    for (int i = 0; i < n; ++i) {
        // strictly inc (change to upper_bound for non-dec):
        auto it = lower_bound(all(tails), a[i]);
        int idx = it - tails.begin();
        if (it == tails.end()) {
            tails.push_back(a[i]);
            tail_indices.push_back(i);
        } else {
            *it = a[i];
            tail_indices[idx] = i;
        }
        if (idx > 0) parent[i] = tail_indices[idx - 1];
    }
    // Reconstruct LIS in O(LIS length):
    vector<int> lis;
    int curr = tail_indices.back();
    while (curr != -1) { lis.push_back(a[curr]); curr = parent[curr]; }
    reverse(all(lis));
    return {(int)tails.size(), lis};
}
```

Longest Common Subsequence (LCS) $O(N \cdot M)$

```
// LCS string in O(N * M) time & space:
string getLCS(const string& s1, const string& s2) {
    int n = s1.size(), m = s2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (s1[i - 1] == s2[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    // Reconstruct str:
    string lcs = "";
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1]) { lcs += s1[i - 1]; i--; j--; }
        else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
        else j--;
    }
    reverse(all(lcs));
    return lcs;
}
```

Maximum Subarray Sum (Kadane's Algorithm) $O(N)$

```
// O(N) time, O(1) space:
ll maxSubarraySum(const vector<ll>& a) {
    ll max_so_far = a[0], cur_max = a[0];
    for (int i = 1; i < sz(a); ++i) {
        cur_max = max(a[i], cur_max + a[i]);
        max_so_far = max(max_so_far, cur_max);
    }
    return max_so_far;
}
```

20. Greedy Algorithms & Paradigms

Fractional Knapsack (Items can be split) $O(N \log N)$

Strategy: Sort items by val/wt desc.

```
struct KnapsackItem { double value, weight; };

// O(N log N) sort + O(N) greedy sweep:
double fractionalKnapsack(double W, vector<KnapsackItem>& items) {
    sort(all(items), [](const KnapsackItem& a, const KnapsackItem& b) {
        return (a.value / a.weight) > (b.value / b.weight);
    });
    double total_val = 0.0;
    for (const auto& item : items) {
        if (W >= item.weight) {
            W -= item.weight;
            total_val += item.value;
        } else {
            total_val += item.value * (W / item.weight);
            break;
        }
    }
    return total_val;
}
```

Interval Scheduling / Activity Selection $O(N \log N)$

Task: Max non-overlapping intervals (sort by end time asc).

```
struct Interval {
    int start, end;
};

// O(N log N) sort + O(N) linear sweep:
int maxNonOverlappingIntervals(vector<Interval>& intervals) {
    if (intervals.empty()) return 0;
    sort(all(intervals), [](const Interval& a, const Interval& b) {
        return a.end < b.end;
    });
    int count = 1, last_end = intervals[0].end;
    for (int i = 1; i < sz(intervals); ++i) {
        if (intervals[i].start >= last_end) {
            ++count;
            last_end = intervals[i].end;
        }
    }
    return count;
}
```

Greedy Coin Change & 2D Kadane Submatrix ($O(\text{count})$ **/** $O(R^2 \cdot C)$**)**

```
// 1. Greedy Coin Change (canonical {25, 10, 5, 1}): O(amount / min_coin)
vector<int> greedyCoinChange(int amount, const vector<int>& coins = {25, 10, 5, 1}) {
    vector<int> result;
    for (int c : coins) { while (amount >= c) { result.push_back(c); amount -= c; } }
    return result;
}

// 2. Maximum Submatrix Sum (2D Kadane) in O(R^2 * C):
ll maxSubmatrixSum(const vector<vector<ll>>& mat) {
    int R = mat.size(), C = mat[0].size(); ll max_sum = -1e18;
    for (int top = 0; top < R; ++top) {
        vector<ll> col_sum(C, 0);
        for (int btm = top; btm < R; ++btm) {
            for (int c = 0; c < C; ++c) col_sum[c] += mat[btm][c];
            ll cur = col_sum[0], best = col_sum[0];
            for (int c = 1; c < C; ++c) {
                cur = max(col_sum[c], cur + col_sum[c]);
                best = max(best, cur);
            }
            max_sum = max(max_sum, best);
        }
    }
    return max_sum;
}
```

21. 2D Matrices, Matrix Exponentiation & Game Theory

2D Matrices: Memory Layout, Compass Deltas & In-Place 90° Rotation ($O(1)$ / $O(N^2)$)

```
// Static: int mat[500][500]; | Dynamic: vector<vector<int>> mat(R, vector<int>(C, 0));
// Cache: Row-major mat[r][c] (outer r, inner c) is contiguous in RAM (L1 hit, 10x faster).
const int dr4[] = {-1, 1, 0, 0}, dc4[] = {0, 0, -1, 1}; // Up, Down, Left, Right
const int dr8[] = {-1, -1, -1, 0, 0, 1, 1, 1}, dc8[] = {-1, 0, 1, -1, 1, -1, 0, 1};
inline bool inBounds(int r, int c, int R, int C) { return r >= 0 && r < R && c >= 0 && c < C; }

// In-place 90 deg clockwise rotation (Transpose + Reverse rows) in O(N^2) time, O(1) space:
void rotateMatrixCW(vector<vector<int>>& mat) {
    int n = mat.size();
    for (int i = 0; i < n; ++i) for (int j = i + 1; j < n; ++j) swap(mat[i][j], mat[j][i]);
    for (int i = 0; i < n; ++i) reverse(mat[i].begin(), mat[i].end());
}
```

Matrix Multiplication ($O(N^3)$) & Binary Matrix Exponentiation ($O(N^3 \log K)$)

```
using Matrix = vector<vector<ll>>;
Matrix matMul(const Matrix& A, const Matrix& B, ll mod = 1e9 + 7) {
    int n = A.size(), m = B[0].size(), p = B.size(); Matrix C(n, vector<ll>(m, 0));
    for (int i = 0; i < n; ++i) for (int k = 0; k < p; ++k) for (int j = 0; j < m; ++j)
        C[i][j] = (C[i][j] + (__int128)A[i][k] * B[k][j]) % mod;
    return C;
}
Matrix matPow(Matrix A, ll p, ll mod = 1e9 + 7) { // O(N^3 log p)
    int n = A.size(); Matrix res(n, vector<ll>(n, 0));
    for (int i = 0; i < n; ++i) res[i][i] = 1; // Identity matrix
    while (p > 0) { if (p & 1) res = matMul(res, A, mod); A = matMul(A, A, mod); p >>= 1; }
    return res;
}
// N-th Fibonacci in O(log N) via Matrix Exponentiation:
ll fibMatrix(ll n, ll mod = 1e9 + 7) {
    if (n <= 0) return 0; if (n == 1) return 1;
    Matrix T = {{1, 1}, {1, 0}}; T = matPow(T, n - 1, mod); return T[0][0];
}
```

Game Theory (Nim Game & Sprague-Grundy Theorem) ($O(|S|)$ Mex)

- **Nim-Sum**: In standard Nim ($x_1, \ldots, x_k$), pos is **winning (P1)** iff $x_1 \oplus \cdots \oplus x_k \neq 0$. If 0, losing (P2).

```
int mex(const unordered_set<int>& s) { int m = 0; while (s.count(m)) ++m; return m; }
```

Traveling Salesperson Problem (TSP Bitmask DP) $O(2^N \cdot N^2)$

```
// O(2^N * N^2) time, O(2^N * N) space:
const int INF = 1e9;
int target = (1 << N) - 1, dp[1 << 18][18];
int tsp(int mask, int u) {
    if (mask == target) return dist[u][0]; // return to start (or 0)
    if (dp[mask][u] != -1) return dp[mask][u];
    int ans = INF;
    for (int v = 0; v < N; ++v) {
        if (!(mask & (1 << v)))
            ans = min(ans, tsp(mask | (1 << v), v) + dist[u][v]);
    }
    return dp[mask][u] = ans;
}
```

XOR Basis / Linear Basis over $GF(2)$ ($O(\log(\max V))$ per Insert)

```
struct XorBasis { // linear basis of a set of integers under XOR: O(60) per insert
    static const int LOG = 60;
    ll basis[LOG] = {}; int closest[LOG]; // closest[j]: idx last used to update basis[j]
    void insert(ll x, int pos = 0) { // O(LOG)
        for (int j = LOG - 1; j >= 0; --j) {
            if (!((x >> j) & 1)) continue;
            if (!basis[j]) { basis[j] = x; closest[j] = pos; return; }
            if (pos > closest[j]) { swap(closest[j], pos); swap(basis[j], x); }
            x ^= basis[j];
        }
    }
    bool canForm(ll x) { // O(LOG): true if x is an XOR of some subset of inserted elems
        for (int j = LOG - 1; j >= 0; --j) if ((x >> j) & 1) { if (!basis[j]) return false; x ^= basis[j];
}
        return true;
    }
};
```

22. 2D Computational Geometry I: Points & Vectors

Complex Numbers as 2D Vector Primitives ($O(1)$)

```
#include <complex>
using ld = double; // or long double
typedef complex<ld> pt;
#define x real()
#define y imag()

// Dot & Cross Prods: O(1)
ld dot(pt a, pt b) { return (conj(a) * b).x; }
ld cross(pt a, pt b) { return (conj(a) * b).y; }

// Dist & Norm: O(1)
ld dist(pt a, pt b) { return abs(a - b); }
ld distSq(pt a, pt b) { return norm(a - b); } // (a.x-b.x)^2 + (a.y-b.y)^2

// Rotations & Angles: O(1)
pt rotate(pt p, ld angle) { return p * polar((ld)1.0, angle); }
pt rotate_around(pt p, pt pivot, ld angle) {
    return pivot + (p - pivot) * polar((ld)1.0, angle);
}
ld angle(pt p) { return arg(p); } // rad in [-pi, pi]
```

Orientation Test (CCW / Turn Direction) ($O(1)$)

- $\text{cross}(b-a, c-a) > 0$: c is **LEFT** of line ab (CCW turn)
- $\text{cross}(b-a, c-a) < 0$: c is **RIGHT** of line ab (CW turn)
- $\text{cross}(b-a, c-a) = 0$: a, b, c are **collinear**

```
int ccw(pt a, pt b, pt c) { // O(1)
    ld cp = cross(b - a, c - a);
    if (abs(cp) < 1e-9) return 0; // collinear
    return (cp > 0) ? 1 : -1;     // +1: CCW (left), -1: CW (right)
}
```

Point Projection & Reflection ($O(1)$)

```
// Project pt p onto line (a, b): O(1)
pt project_on_line(pt p, pt a, pt b) {
    return a + (b - a) * dot(p - a, b - a) / norm(b - a);
}

// Reflect pt p across line (a, b): O(1)
pt reflect_across_line(pt p, pt a, pt b) {
    return a + conj((p - a) / (b - a)) * (b - a);
}

// Dist pt p to line (a, b): O(1)
ld dist_point_to_line(pt p, pt a, pt b) {
    return abs(cross(b - a, p - a)) / abs(b - a);
}

// Dist pt p to seg ab: O(1)
ld dist_point_to_segment(pt p, pt a, pt b) {
    if (dot(p - a, b - a) <= 0) return abs(p - a);
    if (dot(p - b, a - b) <= 0) return abs(p - b);
    return dist_point_to_line(p, a, b);
}

// Convex Hull (Monotone Chain) in O(N log N):
vector<pt> convex_hull(vector<pt> pts) {
    int n = pts.size(), k = 0; if (n <= 2) return pts;
    vector<pt> h(2 * n);
    sort(all(pts), [](pt a, pt b) { return a.x != b.x ? a.x < b.x : a.y < b.y; });
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && ccw(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; --i) {
        while (k >= t && ccw(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}
```

23. 2D Computational Geometry II: Polygons & Lines

Line-Line Intersection & Segment Intersection ($O(1)$)

```
// Intersect line (a, b) & line (c, d): O(1)
pt line_intersection(pt a, pt b, pt c, pt d) {
    ld c1 = cross(c - a, b - a), c2 = cross(d - a, b - a);
    return (c1 * d - c2 * c) / (c1 - c2); // assumes lines not parallel
}

// Check pt q on seg pr (collinear): O(1)
bool onSegment(pt p, pt q, pt r) {
    return q.x <= max(p.x, r.x) && q.x >= min(p.x, r.x) &&
           q.y <= max(p.y, r.y) && q.y >= min(p.y, r.y);
}

// Check seg p1q1 intersects seg p2q2: O(1)
bool segmentsIntersect(pt p1, pt q1, pt p2, pt q2) {
    int o1 = ccw(p1, q1, p2), o2 = ccw(p1, q1, q2);
    int o3 = ccw(p2, q2, p1), o4 = ccw(p2, q2, q1);
    if (o1 != o2 && o3 != o4) return true;
    if (o1 == 0 && onSegment(p1, p2, q1)) return true;
    if (o2 == 0 && onSegment(p1, q2, q1)) return true;
    if (o3 == 0 && onSegment(p2, p1, q2)) return true;
    if (o4 == 0 && onSegment(p2, q1, q2)) return true;
    return false;
}
```

Polygon Area (Shoelace Formula) ($O(N)$)

$$\text{Area} = \frac{1}{2}\left|\sum_{i=0}^{n-1}(x_i y_{i+1} - x_{i+1} y_i)\right|$$

```
// O(N) polygon area:
ld polygonArea(const vector<pt>& p) {
    ld area = 0.0;
    int n = p.size();
    for (int i = 0; i < n; ++i) {
        area += cross(p[i], p[(i + 1) % n]);
    }
    return abs(area) / 2.0;
}
```

Pick's Theorem & Point-in-Polygon ($O(N)$)

- **Pick's Theorem** (for lattice polygons with integer coordinates):

$$\text{Area} = I + \frac{B}{2} - 1 \implies I = \text{Area} - \frac{B}{2} + 1$$

where I = interior lattice points, B = boundary lattice points ($O(N)$).
Boundary points between (x_1, y_1) and (x_2, y_2): $B = \gcd(|x_2 - x_1|, |y_2 - y_1|)$.

```
// Pt in Poly (Ray-Cast): 1 (in), 0 (out), -1 (border) in O(N)
int pointInPolygon(const vector<pt>& p, pt q) {
    int n = p.size(); bool inside = false;
    for (int i = 0; i < n; ++i) {
        pt a = p[i], b = p[(i + 1) % n];
        if (ccw(a, b, q) == 0 && onSegment(a, q, b)) return -1; // on border
        if ((a.y > q.y) != (b.y > q.y)) {
            ld x_inter = a.x + (q.y - a.y) * (b.x - a.x) / (b.y - a.y);
            if (q.x < x_inter) inside = !inside;
        }
    }
    return inside ? 1 : 0;
}

// 3-Point Circumcircle (Center & Radius) in O(1):
pair<pt, ld> circumcircle(pt a, pt b, pt c) {
    pt d = (a - b) * pt(0, 1), e = (a - c) * pt(0, 1);
    pt m1 = (a + b) / 2.0, m2 = (a + c) / 2.0;
    pt center = line_intersection(m1, m1 + d, m2, m2 + e);
    return {center, abs(a - center)};
}
```

24. 2-SAT & Segment Tree

2-SAT via Implication Graph & SCC ($O(N+M)$)

```
// Var i in [0,N): lit 2i = i is TRUE, lit 2i+1 = i is FALSE. Kosaraju-style 2-pass SCC: O(N+M)
struct TwoSat {
    int N; vector<int> order, comp; vector<vector<int>> adj, rev; vector<bool> used, assignment;
    TwoSat(int n) : N(2 * n), adj(N), rev(N), used(N), comp(N, -1), assignment(n, false) {}

    void dfs1(int u) { // O(N+M): topological order by finish time
        used[u] = true;
        for (int v : adj[u]) if (!used[v]) dfs1(v);
        order.push_back(u);
    }
    void dfs2(int u, int id) { // O(N+M): assigns SCC id on reverse graph
        comp[u] = id;
        for (int v : rev[u]) if (comp[v] == -1) dfs2(v, id);
    }
    // Clause (a == na) OR (b == nb); na/nb: whether the literal is negated: O(1)
    void addClause(int a, bool na, int b, bool nb) {
        a = 2 * a ^ na; b = 2 * b ^ nb;
        int notA = a ^ 1, notB = b ^ 1;
        adj[notA].push_back(b); adj[notB].push_back(a);
        rev[b].push_back(notA); rev[a].push_back(notB);
    }
    bool solve() { // O(N+M): false if unsatisfiable, else fills assignment[]
        used.assign(N, false); order.clear();
        for (int i = 0; i < N; ++i) if (!used[i]) dfs1(i);
        comp.assign(N, -1);
        for (int i = 0, j = 0; i < N; ++i) {
            int u = order[N - i - 1];
            if (comp[u] == -1) dfs2(u, j++);
        }
        for (int i = 0; i < N; i += 2) {
            if (comp[i] == comp[i + 1]) return false; // x == not x: contradiction
            assignment[i / 2] = comp[i] > comp[i + 1];
        }
        return true;
    }
};
// Force var i to TRUE: addClause(i, false, i, false)  (i.e. "i OR i")
```

Iterative Segment Tree (Bottom-Up, No Recursion) ($O(N)$ Build, $O(\log N)$ Upd/Query)

```
// Point update + range query (sum by default; change comb() for min/max/gcd): O(log N)
template <class T> struct Seg {
    int n; vector<T> seg; const T ID = 0; // ID: identity elem (0 for sum, INF for min...)
    T comb(T a, T b) { return a + b; }
    void init(int _n) { n = _n; seg.assign(2 * n, ID); } // O(N)
    void pull(int p) { seg[p] = comb(seg[2 * p], seg[2 * p + 1]); }
    void upd(int p, T val) { for (seg[p += n] = val; p /= 2; ) pull(p); } // O(log N)
    T query(int l, int r) { // O(log N): [l, r] 0-idx inclusive
        T ra = ID, rb = ID;
        for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
            if (l & 1) ra = comb(ra, seg[l++]);
            if (r & 1) rb = comb(seg[--r], rb);
        }
        return comb(ra, rb);
    }
};
```